

Assignment 12: Instruction Pipelining and RISC vs. CISC

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 5, 2026

This assignment follows Week 12's treatment of pipeline timing, hazards, workload-based design, and RISC/CISC trade-offs [2, 1].

Answer every question. Show all working for Numerical and Design questions. State the assumptions that make each shortcut valid.

12.1 Conceptual

Q1. A five-stage pipeline has stages IF, ID, EX, MEM, and WB. Explain the difference between the latency of one instruction and the steady-state throughput of an independent instruction stream. Then explain why pipeline registers and the slowest stage constrain the clock period, and name two effects that can keep a finite stream below its ideal speedup.

Q2. The operation $A[i] = A[i] + 1$ can be represented by the register-centered sequence `LOAD R1, [R2+R3]; ADD R1, R1, 1; STORE [R2+R3], R1`, or by a memory-rich instruction such as `INC [R2+R3]`. Compare the pipeline and implementation consequences of these two tendencies. Explain why the statement “RISC always has fewer instructions and CISC always has lower CPI” is too strong, and identify the measure that should decide the comparison for a fixed workload.

12.2 Numerical

Q3. Eight independent instructions execute in a balanced five-stage pipeline, with one cycle per stage, no hazards, and no extra pipeline-register cost.

- (a) Draw the complete timing table through the WB of the eighth instruction.
- (b) Compute the sequential time, ideal pipelined time, and finite- stream speedup.
- (c) State the limiting ideal speedup as the stream becomes very long. Distinguish this limit from one-instruction latency and from steady-state throughput.

Q4. Consider the four-instruction stream

`LW R4, 8(R5);    ADD R6, R4, R7;    SUB R8, R6, R9;    OR R10, R11, R12.`

Use the same five-stage model. Assume that load data becomes available after MEM, the immediately dependent ALU instruction needs it at the start of EX, and ALU results can be forwarded. Draw the timing table for cycles 1 through 9, mark the bubble, identify both RAW dependences, and state which value is forwarded on each dependence.

12.3 Design

Q5. Two implementations execute the same workload. The RISC-like implementation executes 1.50×10^9 instructions at $\text{CPI} = 1.2$ and $f = 2.5$ GHz. The CISC-like implementation executes 0.90×10^9 instructions at $\text{CPI} = 2.1$ and $f = 3.0$ GHz.

- (a) Compute both CPU times using $T_{\text{CPU}} = \text{IC} \times \text{CPI}/f$.
- (b) Identify the faster implementation for this workload and compute its percentage time advantage relative to the slower one.
- (c) Explain why the higher CPI does not by itself determine the result, and name two workload or system changes that could reverse it.

Q6. Write a design recommendation for each workload below. For each recommendation, select the more helpful tendency (regular, register-centered RISC or richer, memory-oriented CISC), justify it with at least two entries from the Week 12 trade-off discussion, identify one risk or cost, and name the measurements that should be collected before making a final decision.

- (a) Embedded control: tight power budget, predictable loops, limited memory, and fixed sensor operations.
- (b) Cloud analytics: a large software base, diverse branches, high throughput, and frequent vector or multicore parallelism.

Conclude by explaining why neither recommendation is a universal ranking of RISC and CISC.

References

- [1] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.
- [2] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.